

Table of contents

Algorithmes évolutionnaires	2
Introduction	2
Idée clé	3
Types d'algorithmes évolutionnaires	3
Caractéristiques clés des GA	3
Algorithmes génétiques (GA)	3
Maximisation de la fitness	4
Processus itératif	4
Élitisme	4
Pseudocode d'un GA	5
Résumé du processus évolutif global	5
Initialisation et critère d'arrêt	5
Étape de sélection	6
Mécanisme de sélection	6
Étape de croisement	7
Application du croisement	7
Étape de mutation	7
Exploration vs Exploitation	8
Exemple jouet	8
Population initiale	8
Étape de sélection (proportionnelle à la fitness)	9
Étape de croisement	10
Étape de mutation	12
Nouvelle génération à $t + 1$	13
Exemple plus réaliste	13
Évolution de la fitness	13
Diversité de la population	14
Cas d'une fonction continue réelle	14
Fonctions de test Rastrigin et Schwefel	15
Autre exemple	15
Le défi de l'encodage	15
Description d'une exécution GA	16
Notes sur l'encodage	17
Encodage en virgule flottante des nombres réels	17
GA avec nombres réels	18
Autres types d'opérateurs de sélection	18
Révision de la sélection proportionnelle (à la fitness)	18
Sélection par rang	19
Sélection par tournoi	20

Temps de prise de contrôle (Takeover time) : Intensité de sélection	20
Estimation pratique	21
Solution de l'équation logistique	22
Autres approches de croisement et de mutation	22
Autres opérateurs de croisement	23
Autres opérateurs de mutation	24
Utilité des opérateurs génétiques	24
Pourquoi le croisement est efficace ?	25
Utilité de l'opérateur de croisement avec une fonction réelle	25
Exemple : Couverture optimale de zone par antennes GSM	26
Théorème du schéma (Schema Theorem)	26
Définitions	26
Ordre et longueur d'un schéma	27
Fitness d'un schéma S	27
Évolution du nombre moyen $M(t, S)$	28
Inégalité fondamentale	29
Théorème du schéma de Holland	29
Population structurée	30
Multi-populations	30
Populations cellulaires	32
Temps de prise de contrôle avec populations structurées	32

Algorithmes évolutionnaires

Introduction

Métaheuristique stochastique **basée sur une population** introduite dans les années 70 (parmi les toutes premières).

Famille de métaheuristiques basée sur le concept de l'évolution darwinienne : **sélection naturelle** ou "survie du plus apte".

Les individus "meilleurs" se reproduisent (sélection) et évoluent.

Composants principaux :

- **Sélection**
- **Mutation**
- **Croisement** (crossover) des individus les plus aptes.

Idée clé

Simuler le processus d'évolution pour faire émerger des solutions de haute qualité pour un problème donné : recherche de bonnes solutions en faisant évoluer une population de solutions candidates via la variation et la sélection.

Types d'algorithmes évolutionnaires

- **Algorithmes génétiques (GA)** – John Holland
 - **Stratégie d'évolution (ES)** – Ingo Rechenberg
 - **Programmation génétique (GP)** – Lawrence J. Fogel, John R. Koza
-

Caractéristiques clés des GA

- **Basés sur une population** : explore de nombreuses régions de l'espace de recherche simultanément.
 - **Opérateurs probabilistes** : utilisent l'aléatoire mais sont guidés par la fitness.
 - **Capacité de recherche globale** : moins sujet aux minima locaux que les méthodes purement déterministes.
 - **Optimisation boîte noire** : ne nécessite pas de dérivées ou d'informations de gradient.
-

Algorithmes génétiques (GA)

Nous considérons une population d'individus.

Chaque individu correspond à une solution (c'est-à-dire un point dans l'espace de recherche S).

Comme pour les autres métaheuristiques, le choix de la **représentation** de la solution joue un rôle important. Pour les GA, nous parlons d'**encodage** du jeu fictif de chromosomes/le génotype de chaque individu (le phénotype).

- **Encodage binaire** : par exemple pour le problème MaxOne.
- **Encodage à valeurs réelles** : pour l'optimisation continue.
- **Encodage par permutation** : pour les problèmes d'ordonnancement (ex. : TSP).

Le niveau d'adaptation de chaque individu est naturellement donné par sa **fitness**.

Maximisation de la fitness

Dans ce cours, nous nous concentrons sur la **maximisation** de la fitness.

En biologie, la fitness peut être mesurée par le nombre de descendants → plus le nombre de descendants est grand, plus l'espèce est adaptée.

Processus itératif

Initialisation : L'évolution commence à partir d'une population d'individus générés aléatoirement.

Suivi d'un processus itératif :

La population à chaque itération est appelée une **génération**.

À chaque génération, la fitness de chaque individu de la population est évaluée. Ensuite, on effectue :

1. **Sélection** des individus les plus aptes dans toute la population.
 2. **Mutations** (transformation aléatoire du génotype).
 3. **Croisement** (hybridation du génotype d'une paire d'individus).
-

Élitisme

Pour limiter l'impact de l'exploration et promouvoir l'exploitation, et afin de ne pas perdre la meilleure solution trouvée jusqu'à présent, celle-ci est automatiquement insérée dans la dernière génération. C'est le processus dit d'**élitisme**.

L'élitisme est implémenté en remplaçant l'individu le moins apte par le plus apte (non nécessaire si le plus apte est déjà apparu au cours de la génération en cours).

La taille de la population est maintenue constante tout au long de l'évolution.

Pseudocode d'un GA

```
génération = 0
initialiser la population

tant que (condition d'arrêt non atteinte)
    génération += 1
    évaluer : calculer la fitness de chaque individu
    sélectionner des individus
    croisement
    mutation
    insérer le meilleur
fin tant que

afficher le meilleur
```

Résumé du processus évolutif global

$$P(t) \xrightarrow{\text{sélection}} P'(t) \xrightarrow{\text{croisement}} P''(t) \xrightarrow{\text{mutation}} P'''(t) = P(t+1)$$

Le maintien d'une taille de population constante a des implications importantes sur la définition des opérateurs de sélection et de croisement.

Initialisation et critère d'arrêt

Population initiale :

- Taille n : contient typiquement des dizaines à quelques centaines d'individus.
- Par défaut, générée aléatoirement, à moins qu'une connaissance préalable ne soit disponible.

Critère d'arrêt :

- Par défaut après un nombre donné d'itérations, ou en l'absence d'amélioration de la fitness.
 - Retourne le meilleur individu de la population.
-

Étape de sélection

$$P(t) \xrightarrow{\text{sélection}} P'(t)$$

La sélection choisit des individus de la population actuelle pour devenir parents de la génération suivante.

Survie du plus apte : principe de proportionnalité à la fitness. Les individus ayant une fitness plus élevée ont une probabilité plus élevée d'être sélectionnés pour se reproduire.

Une certaine stochasticité est préservée : des individus moins aptes peuvent parfois se reproduire aussi, ce qui aide à maintenir la diversité et à éviter une convergence prématurée.

Pression de sélection : mesure à quel point le processus de sélection favorise les meilleurs individus.

- Pression de sélection élevée → convergence rapide mais risque de convergence prématurée (perte de diversité).
 - Pression de sélection faible → meilleure diversité mais convergence plus lente.
-

Mécanisme de sélection

Nous tirons, **avec remise**, n fois un individu parmi la population courante de taille n .

Le choix est effectué aléatoirement mais implique la fonction de fitness. Par exemple, la probabilité de sélectionner l'individu k est proportionnelle à sa fitness.

D'autres méthodes sont possibles : sélection par tournoi ou par rang (voir plus tard).

À la fin de l'étape de sélection, nous avons une population temporaire dans laquelle un individu donné peut apparaître plusieurs fois. Par construction, il est attendu que cette population contienne les individus les plus aptes de la population originale.

Étape de croisement

$$P'(t) \xrightarrow{\text{croisement}} P''(t)$$

Les n individus issus de l'étape de sélection constituent les parents de la génération suivante obtenue après croisement et mutation.

Croisement :

- Conçu pour mélanger l'information de deux parents afin de créer une descendance qui combine leurs traits avantageux.
 - Mécanisme essentiel qui donne aux GA leur puissance : construire de nouvelles solutions à partir de bonnes composantes existantes.
-

Application du croisement

Nous formons des paires d'individus basées sur les parents identifiés lors de l'étape de sélection. Par exemple, les paires (s_i, s_{i+1}) , où s_i est le i -ème individu, $i = 1, 3, \dots$

Avec une probabilité donnée $p_{\text{croisement}}$, nous mélangeons l'encodage (génome) des deux parents pour générer le génome de deux descendants les remplaçant.

Avec une probabilité $1 - p_{\text{croisement}}$, le croisement n'est pas appliqué : les descendants sont des copies exactes des parents.

Il existe différents opérateurs de croisement (voir plus tard).

Étape de mutation

$$P''(t) \xrightarrow{\text{mutation}} P'''(t) = P(t+1)$$

La mutation est la principale source de nouveauté génétique et est essentielle pour maintenir l'exploration.

Une mutation peut être appliquée aux n nouveaux individus générés par l'étape de croisement précédente, avec une probabilité p_{mutation} .

Cette étape de mutation est destinée à imiter les erreurs de transcription et le bruit exogène qui peuvent créer de nouvelles solutions → exploration supplémentaire.

Une mutation consiste à modifier aléatoirement l'encodage/génome d'un individu donné.

Exploration vs Exploitation

- L'opérateur de **sélection** favorise l'intensification $\rightarrow$ **exploitation**.
- L'opérateur de **mutation** favorise la diversification $\rightarrow$ **exploration**.
- L'opérateur de **croisement** favorise également plus la diversification que l'intensification (les deux ne sont pas mutuellement exclusifs).

Paramètres de guidage :

- Taille de la population n .
 - Probabilité de mutation P_{mutation} .
 - Probabilité de croisement $P_{\text{croisement}}$.
-

Exemple jouet

Illustrons les étapes clés d'un GA avec un problème basique : **maxOne**, un problème particulier de paysage NK avec $K = 0$.

But : trouver la chaîne de bits de longueur ℓ telle que le nombre de bits à 1 soit maximum.

La solution est évidente pour un humain : $x = 111 \dots 111$, mais c'est un défi pour le GA de l'identifier. Pour illustrer, nous considérons $\ell = 10$ et une taille de population $n = 6$.

Population initiale

Supposons que le tirage aléatoire initial donne :

- $s_1 = 1111010101 \quad f(s_1) = 7$
- $s_2 = 0111000101 \quad f(s_2) = 5$
- $s_3 = 1110110101 \quad f(s_3) = 7$
- $s_4 = 0100010011 \quad f(s_4) = 4$
- $s_5 = 1110111101 \quad f(s_5) = 8$
- $s_6 = 0100110000 \quad f(s_6) = 3$

où la fitness f est simplement définie comme :

$$f(s_i) = f(x_1 x_2 \dots x_\ell) = \sum_{j=1}^{\ell} x_j$$

Étape de sélection (proportionnelle à la fitness)

Nous considérons un processus de sélection proportionnel à la fitness :

La probabilité p_i de sélectionner l'individu s_i est :

$$p_i = \frac{f(s_i)}{\sum_{k=1}^n f(s_k)}$$

Dans ce problème, la somme des fitness est 34 et sa valeur moyenne est $34/6 = 5.67$.

Ainsi, la probabilité de sélectionner s_1 est $7/34 = 0.2059$ tandis que s_6 sera sélectionné avec une probabilité $3/34 = 0.088$.

Roulette (Roulette wheel)

Nous tirons un nombre aléatoire $r = \text{rand}(0, 1)$ et nous sélectionnons l'individu i tel que :

$$\sum_{k=1}^{i-1} p_k < r \leq \sum_{k=1}^i p_k$$

Table des probabilités cumulées :

Individu	Probabilité	Intervalle cumulatif
s_1	0.2059	0.0000 – 0.2059
s_2	0.1471	0.2059 – 0.3530
s_3	0.2059	0.3530 – 0.5589
s_4	0.1176	0.5589 – 0.6765
s_5	0.2353	0.6765 – 0.9118
s_6	0.0882	0.9118 – 1.0000

Résultat de la sélection

Supposons qu'à la fin de l'étape de sélection (6 tirages avec remplacement), nous obtenions la population temporaire suivante :

- $s'_1 = 1111010101$ (copie de s_1)
- $s'_2 = 1110110101$ (copie de s_3)
- $s'_3 = 1110111101$ (copie de s_5)
- $s'_4 = 0111000101$ (copie de s_2)
- $s'_5 = 0100010011$ (copie de s_4)
- $s'_6 = 1110111101$ (copie de s_5)

Note : La chaîne s_5 a été sélectionnée deux fois (fitness la plus élevée). La chaîne s_6 n'a jamais été sélectionnée (fitness la plus basse).

L'étape de sélection donne plus de chances à certains individus à haute fitness de se retrouver dans la prochaine génération. Évidemment, il est crucial d'introduire un élément d' "innovation" via l'exploration, sinon aucune nouvelle solution ne peut émerger. Ces variations exploratoires sont réalisées par les opérateurs génétiques de croisement et de mutation.

Étape de croisement

L'opérateur de croisement doit maintenant être spécifiquement défini. La chaîne binaire peut être vue comme un code génétique (encodage) avec 2 symboles (0 et 1) au lieu des 4 symboles d'ADN (A, C, G, T).

Croisement à un point :

Il consiste à créer un nouvel individu en prenant les premiers k "gènes" du premier parent et les gènes de $k + 1$ à ℓ du second parent.

Ici, k est le point de croisement choisi aléatoirement entre 1 et $\ell - 1$.

Pour obtenir le deuxième descendant : nous prenons les gènes $1 : k$ du second parent croisés avec les gènes $(k + 1) : \ell$ du premier parent. Par ce processus, à partir de deux parents, nous obtenons deux descendants.

Application au sein de la population

Pour appliquer l'opérateur de croisement à toute la population, nous formons des paires (s_i, s_{i+1}) .

Avec $n = 6$ chaînes/individus, s'_1 est mélangé avec s'_2 , s'_3 avec s'_4 , et s'_5 avec s'_6 , chaque fois avec une probabilité de croisement $p_{\text{croisement}}$.

Choisissons $p_{\text{croisement}} = 0.6$. Supposons qu'après tirage des nombres pseudo-aléatoires, seules les paires (s'_1, s'_2) et (s'_5, s'_6) sont hybridées.

Premier croisement

Supposons que le point de croisement soit $k = 2$ pour la première paire. Avant le croisement :

$$s'_1 = 11 \cdot 11010101$$

$$s'_2 = 11 \cdot 10110101$$

Purement par hasard, dans ce cas particulier, le croisement n'a produit aucune nouvelle chaîne puisque les descendants sont identiques aux parents. Cela arrive souvent lorsque les parents sont similaires $\rightarrow$ exploitation du GA.

Après le croisement :

$$s''_1 = 11 \cdot 10110101$$

$$s''_2 = 11 \cdot 11010101$$

Deuxième croisement

Pour la paire (s'_5, s'_6) , supposons $k = 5$. Avant croisement :

$$s'_5 = 01000 \cdot 10011$$

$$s'_6 = 11101 \cdot 11101$$

Après croisement :

$$s''_5 = 01000 \cdot 11101$$

$$s_6'' = 11101 \cdot 10011$$

Cette fois, le croisement a produit des descendants distincts des parents. Nous avons généré de la diversité → exploration.

Étape de mutation

La dernière étape pour générer une nouvelle population implique une mutation aléatoire d'un ou plusieurs gènes de la population de descendants générée par croisement.

Pour chaque chaîne/individu ET pour chaque chiffre binaire de la chaîne, nous créons la possibilité de l'inverser avec une petite probabilité, disons 0.1.

Cette valeur implique que sur 60 chiffres binaires au total ($60 = 6 \times 10$), nous attendons qu'en moyenne 6 soient inversés.

À la fin de ce processus, la génération résultante pourrait ressembler à ceci (les chiffres à inverser sont surlignés) :

- $s_1'' = 111 \text{ 011 0101}$ (aucun changement)
- $s_2'' = 111 \text{ 101 0101}$ (aucun changement)
- $s_3'' = 111 \text{ 011 1101}$ (aucun changement)
- $s_4'' = 011 \text{ 100 0101}$ (aucun changement)
- $s_5'' = 010 \text{ 001 1101}$ (aucun changement)
- $s_6'' = 111 \text{ 011 0011}$ (le dernier 1 devient 0)

Sur les 6 mutations potentielles, 4 transforment un '1' en '0', réduisant ainsi la fitness de la chaîne/individu → dégradation de la fitness.

Ce n'est pas surprenant, étant donné que les "bonnes" solutions tendent à prévaloir à ce stade, donc les mutations vont probablement détériorer le résultat, mais occasionnellement elles peuvent l'améliorer → exploration.

Nouvelle génération à $t + 1$

Une fois l'étape de mutation terminée, nous avons la nouvelle génération suivante :

$$\begin{array}{ll} s_1^{HH} = 111\ 010\ 0101 & f(s_1^{HH}) = 6 \\ s_2^{HH} = 111\ 111\ 0100 & f(s_2^{HH}) = 7 \\ s_3^{HH} = 111\ 010\ 1111 & f(s_3^{HH}) = 8 \\ s_4^{HH} = 011\ 100\ 0101 & f(s_4^{HH}) = 5 \\ s_5^{HH} = 010\ 001\ 1101 & f(s_5^{HH}) = 5 \\ s_6^{HH} = 111\ 011\ 0001 & f(s_6^{HH}) = 6 \end{array}$$

En l'espace d'une seule génération, la **fitness totale** est passée de 34 à 37, soit une augmentation de 7%.

La fitness moyenne est passée de 5.666 à 6.166.

Ce processus itératif pourrait continuer jusqu'à ce qu'un critère d'arrêt le stoppe.

Exemple plus réaliste

Considérons maintenant l'évolution d'une population "réelle" de n individus (c'est-à-dire n chaînes binaires).

Les chaînes ont maintenant une longueur $\ell = 128$.

Le GA s'arrête lorsqu'il trouve la chaîne optimale ou après 100 itérations.

Nous choisissons $p_{\text{croisement}} = 0.8$ et $p_{\text{mutation}} = 0.05$.

Évolution de la fitness

Sur un run particulier du GA appliqué au problème MaxOne, on observe :

- Cette exécution particulière a découvert la chaîne optimale avec une fitness de 128.
- La fitness moyenne vers la fin de l'exécution est extrêmement proche de l'optimum de 128, ce qui indique que la majorité de la population est proche de l'optimum global.
- L'augmentation de la fitness (du meilleur et de la moyenne) est rapide au début, puis plafonne → Comportement classique de **diminishing returns**.

Cette observation tend à être assez générale pour les algorithmes évolutionnaires en général et les GA en particulier : la population globale s'améliore mais il tend à devenir de plus en plus difficile d'augmenter sa fitness.

Diversité de la population

La diversité est maximale initialement puis diminue au fil des générations. Cette tendance n'est pas surprenante : la population initiale est choisie aléatoirement, donc présente une diversité maximale.

Lors de l'itération à travers les générations, le processus de sélection tend à homogénéiser la population pour faire émerger le caractère de haute fitness.

Les opérateurs de croisement et de mutation tendent à contrebalancer cet effet d'homogénéisation. Cependant, ces deux opérateurs ne peuvent pas contrebalancer la perte de diversité induite par le processus de sélection.

Le but de l'algorithme est de converger vers de "bonnes" solutions, qui par construction partagent de nombreuses similitudes.

Cas d'une fonction continue réelle

Les EA tendent à être très efficaces pour optimiser des problèmes avec des fonctions de fitness multivariées et multimodales difficiles.

Cette classe de métaheuristiques peut facilement éviter de rester piégée dans des optima locaux grâce au croisement et à la mutation.

Un autre avantage majeur est que les EA ne nécessitent pas de dérivées de la fonction de fitness, ils peuvent donc facilement être appliqués à :

- Des fonctions de fitness discontinues.
- Des fonctions de fitness variant dans le temps.

Cela peut être vérifié en utilisant les fonctions de test de référence bien connues de **Rastrigin** et **Schwefel**, qui peuvent avoir des dizaines de variables et ont été conçues pour défier les optimiseurs classiques.

Fonctions de test Rastrigin et Schwefel

$$f_R(x) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)] \quad A = 10$$

$$f_S(x) = Bn - \sum_{i=1}^n x_i \sin(\sqrt{|x_i|}) \quad B = 418.983$$

Autre exemple

Considérons la fonction à valeurs réelles ($\mathbb{R} \rightarrow \mathbb{R}$) suivante :

$$f(x) = - |x \sin(\sqrt{|x|})| + C$$

Ce problème apparaît trompeusement simple car il peut être résolu “à la main”. Cependant, il est utilisé pour décomposer le fonctionnement interne des GA avant d’aborder des cas plus complexes et réalistes.

Nous cherchons x^* dans l’intervalle $D = [-512, 512]$ tel que $f(x^*)$ atteigne sa valeur minimale. Comme $f(x)$ est symétrique, il suffit d’étudier le problème pour les valeurs positives de $x : x \in \mathbb{R}^+$.

Le défi de l’encodage

Une solution admissible de ce problème est toute valeur de x dans l’intervalle $[0, 512]$. Un individu au sein de la population doit être encodé sous la forme d’une chaîne binaire avec une longueur appropriée ℓ afin de représenter correctement la valeur d’un nombre réel.

Ainsi, la valeur de ℓ dépend de la précision recherchée : plus ℓ est long, plus la précision est élevée.

Contrairement au problème MaxOne (solutions intrinsèquement binaires), nous devons décoder les chaînes de binaires en nombres réels afin de pouvoir évaluer la fonction.

Exemple d'encodage

Par exemple, si chaque individu x est représenté par une chaîne binaire de 10 bits, alors $2^{10} = 1,024$ valeurs au total sont disponibles pour couvrir l'intervalle $[0, 512]$.

$$x = \frac{512}{1023}s, \quad x \in \mathbb{R}, \quad s \in \{0, \dots, 1023\}$$

Nous aurons donc une granularité d'environ 0.5 : nous pouvons échantillonner des points avec un espacement d'environ 0.5.

Les chaînes 0000000000 et 1111111111 représentent respectivement les bornes inférieure et supérieure de l'intervalle de recherche. Toute autre chaîne sera à l'intérieur de l'intervalle.

Description d'une exécution GA

Nous considérons une population de $n = 50$ individus. La solution optimale est $x^* = 421.016$ avec $f(x^*) = 0.0176086$. Il ne faut pas oublier l'erreur de précision associée à l'encodage choisi.

Génération	Meilleure fitness	Fitness moyenne
0	104.302	68.703
5	52.677	8.619
9	0.00179	32.711
18	0.00179	14.322
26	0.00179	5.833
36	0.00179	2.725
50	0.00179	1.776
90	0.00179	0.15

Compte tenu de la précision, la solution optimale est déjà trouvée à la 9ème génération.

Avec cet exemple, l'optimisation ne s'est pas coincée dans l'un des nombreux minima locaux. Les GA semblent efficaces avec de tels paysages de fitness multimodaux.

La fitness moyenne continue de s'améliorer au fil des générations, montrant ainsi que toute la population converge vers l'optimum.

Notes sur l'encodage

Pour l'optimisation continue (fonctions $\mathbb{R}^d \rightarrow \mathbb{R}$), l'encodage précédemment considéré n'est pas toujours approprié.

Pour une chaîne binaire de longueur fixe, la précision de la solution dépendra de la taille de l'intervalle de recherche.

Un GA tel que celui décrit précédemment ne sera pas efficace avec ce type d'espace de recherche. Nous recommandons donc de considérer un encodage en **virgule flottante**.

Encodage en virgule flottante des nombres réels

Tout nombre réel x peut s'écrire $s \times r^t$ où s est la mantisse, t l'exposant et r la base.

Par exemple, avec $r = 2$ (binaire) :

$$x = \left(b_0 + \sum_{k=1}^{p-1} b_k 2^{-k} \right) \times 2^t \quad t = \sum_{k=p}^{n-1} b_k 2^{k+1-p}$$

La représentation informatique des nombres à virgule flottante utilise $r = 2$ et p bits pour encoder s et le reste, $q = n - p$ bits pour encoder l'exposant t .

p est la précision d'encodage.

La norme IEEE 754 propose la double précision (64 bits) avec : 1 bit pour le signe, $p = 52$ bits pour la mantisse et $q = 11$ bits pour l'exposant.

Cette norme donne environ 16 chiffres décimaux significatifs et un exposant entre -1024 et $+1023$.

En simple précision (32 bits) : $p = 23$ (7 chiffres décimaux significatifs) et $q = 8$ bits pour l'exposant (-128 à $+127$).

GA avec nombres réels

On peut choisir la précision requise en fonction de ses besoins spécifiques, y compris le nombre q de bits pour l'exposant.

Les opérateurs de croisement et de mutation sont appliqués indépendamment à la mantisse et à l'exposant, avec des probabilités potentiellement différentes.

Cela permet donc de rechercher dans des espaces de recherche complexes tels que ceux des types Rastrigin et Schwefel.

NB : Il est également possible de formuler directement un GA avec des nombres réels (voir l'évolution stratégique au chapitre 7).

Autres types d'opérateurs de sélection

$$P(t) \xrightarrow{\text{sélection}} P'(t) \xrightarrow{\text{croisement}} P''(t) \xrightarrow{\text{mutation}} P'''(t) = P(t+1)$$

Sélection déterministe : Selon un critère donné qui sélectionne ou rejette un individu de la population.

Sélection stochastique : Se produit sur la base d'un tirage aléatoire. Elle a l'avantage de permettre à des individus moins aptes de rejoindre la population (équivalent en quelque sorte à une dégradation de la fitness).

Révision de la sélection proportionnelle (à la fitness)

La soi-disant "roulette" : $p_i = \frac{f_i}{\sum_j f_j}$

Pour les problèmes de maximisation (sinon redéfinir f avec $-f$). La fitness doit être positive sinon ajouter une constante suffisamment grande f_0 .

L'évolution tend à réduire la diversité de la population : les f_i tendent à se rapprocher les unes des autres. Cela a un impact négatif : la sélection proportionnelle devient moins sélective.

Pour augmenter la sélectivité, on peut envisager de mettre en œuvre une approche proportionnelle à une fitness modifiée $\bar{f}$:

$$\bar{f}(t) = f - \min_{i \in P(t)} f_i$$

L'objectif est d'amplifier les différences entre les différents individus à une génération donnée.

Sélection par rang

Les limitations de la sélection proportionnelle peuvent être surmontées avec une autre approche : sensibilité aux valeurs aberrantes, mise à l'échelle, difficulté à maintenir la diversité, localité limitée.

Nous considérons un **classement** de la population basé sur la fitness (minimisation ou maximisation de f).

Nous proposons de sélectionner l'individu de rang supérieur, ignorant la valeur réelle de sa fitness.

Sélection par rang linéaire

La probabilité p_i de sélectionner l'individu i s'écrit :

$$p_i = \frac{1}{n} \left[\beta - 2(\beta - 1) \frac{i - 1}{n - 1} \right], \quad i = 1, \dots, n$$

où l'indice i est le rang de l'individu après tri.

β est un paramètre qui représente le nombre de copies attendues pour le meilleur individu (avec rang 1 et remplacement). $\beta > 1$ est requis pour que $p_1 > p_n$ et $\beta \leq 2$ pour avoir $p_n > 0$.

On peut facilement vérifier que $\sum_{i=1}^n p_i = 1$.

L'intensité de la sélection (ou sélectivité) est réglée via β : plus β est grand, plus la sélectivité est forte. Avec $\beta = 1$, tous les individus sont équiprobables.

Il existe également des méthodes non linéaires basées sur le rang.

Sélection par tournoi

Le **tournoi le plus basique** possible est un tirage aléatoire uniforme de $k = 2$ individus au sein de la population $P(t)$.

- Ces 2 individus sont ensuite comparés, par exemple selon leurs fitness respectives.
- Le gagnant (fitness plus élevée) est sélectionné et placé dans la population intermédiaire $P'(t)$.
- Les deux candidats sont remis dans $P(t)$ pour potentiellement participer aux prochains tours de tournoi.
- Nous exécutons n de ces tours de tournoi pour compléter entièrement $P'(t)$.

Nous pouvons également considérer des tournois avec plus d'individus tant que $k \ll n$. Par exemple $k \in \{2, \dots, 10\}$.

Avantages de la sélection par tournoi

- Contrôle facile et précis de la pression de sélection.
- Ajustement de la pression sélective simplement en changeant la taille du tournoi k . Cas extrêmes : $k = 1$ et $k = n$. Ce réglage est explicite et prévisible, contrairement à la sélection proportionnelle (FPS) où la pression dépend de toute la distribution de fitness.
- Couramment utilisé : fonctionne naturellement avec toute distribution de fitness.
- La sélection par tournoi ne nécessite pas de fitness non négative, de mise à l'échelle ou de normalisation. Résistante aux valeurs aberrantes.
- Facile à implémenter.
- Un autre avantage : il suffit de pouvoir comparer la performance relative des individus, sans autre mesure de "qualité". Par exemple, en robotique, il suffit de comparer les performances du robot sur une tâche donnée, sans avoir besoin de définir une fonction de fitness.

Temps de prise de contrôle (Takeover time) : Intensité de sélection

Le **temps de prise de contrôle** τ est un concept théorique défini comme le nombre de tours de sélection requis (sans croisement ni mutation) pour que le meilleur individu prenne le contrôle de toute la population.

C'est une mesure de l'intensité de la sélection :

- Si τ est petit, la sélection est forte.
- Si τ est grand, nous avons plutôt une faible intensité de sélection.

Par exemple, si à chacun des n tours du processus de sélection, nous mettons le meilleur individu dans P' , alors $\tau = 1$.

En pratique, τ est défini comme le temps nécessaire pour avoir $n/2$ copies du meilleur individu.

Estimation pratique

En pratique, le temps de prise de contrôle τ peut être estimé par une exploration computationnelle.

On peut s'attendre à ce que le nombre de copies $m(t)$ du meilleur individu après t sélections suive une équation correspondant à la **carte logistique** :

$$\frac{dm}{dt} = \alpha m \left(1 - \frac{m}{n}\right)$$

Avec sélection seulement, le nombre $m(t)$ de copies du meilleur individu évolue comme :

$$m(t+1) = n \times \text{Probabilité(sélectionner le meilleur individu)}$$

La probabilité de tirer le meilleur individu est proportionnelle à m/n (qui est la fraction de copies existantes dans la population) multipliée par un coefficient α qui dépend de la fitness.

Ce coefficient diminue comme la fraction de places restantes $1 - m/n$.

La moyenne m varie comme :

$$n \times \left(\alpha \frac{m}{n} \left(1 - \frac{m}{n}\right) \right)$$

Solution de l'équation logistique

$$m(t) = \frac{n}{1 + \frac{n-m_0}{m_0} e^{-\alpha t}} \quad (1)$$

où $m_0 = m(t=0) = 1$.

Avec

$$\tau = \frac{\ln(n-1)}{\alpha} \quad (2)$$

nous obtenons

$$m(\tau) = \frac{n}{2} \quad (3)$$

Cela donne le temps de prise de contrôle $\tau = \mathcal{O}(\ln n)$.

Autres approches de croisement et de mutation

Nous avons introduit le **croisement à un point** de base.

Nous pouvons également considérer un **croisement à deux points**, dans lequel les deux descendants sont :

$$\begin{aligned} & a_1 a_2 \dots a_k b_{k+1} \dots b_m a_{m+1} \dots a_\ell \\ & b_1 b_2 \dots b_k a_{k+1} \dots a_m b_{m+1} \dots b_\ell \end{aligned}$$

Une autre approche est le **croisement uniforme** :

Avec probabilité $1/2$, on choisit le bit du premier parent, ou celui du second parent. Dans ce cas, les bits identiques chez les deux parents ne peuvent pas être modifiés.

Le concept derrière ce croisement est d'exploiter le fait que dans certains cas, les premiers bits d'un individu sont bons, mais pas les suivants, alors que l'inverse est vrai pour un autre individu.

Autres opérateurs de croisement

Avec certains espaces de recherche S , les opérateurs de croisement précédemment considérés ne sont pas efficaces car ils produisent des individus non viables.

C'est particulièrement vrai si S est l'espace des permutations (par exemple, TSP et QAP).

Exemple de croisement entre 2 cycles parmi 8 villes :

$$\begin{array}{c} 1 - 2 - 4 - | - 3 - 8 - 5 - 6 - 7 \\ 1 - 3 - 4 - | - 8 - 2 - 5 - 7 - 6 \end{array}$$

Si nous utilisons le croisement à un point régulier, cela donnerait une solution non admissible :

$$1 - 2 - 4 - 8 - 2 - 5 - 7 - 6$$

Si l'on insiste pour un croisement des deux cycles T_1 et T_2 , l'approche suivante donne un descendant admissible :

$$\begin{array}{l} T_1 = 1 - 4 - | 3 - 5 - 2 | - 6 - 7 - 8 \\ T_2 = 1 - 3 - 2 - 6 - 4 - 5 - 7 - 8 \end{array}$$

La section à deux points sélectionnée aléatoirement (entre les barres verticales) du parent T_1 est transférée au descendant T_3 .

Ensuite, les villes du parent T_2 qui n'ont pas encore été utilisées sont insérées dans le même ordre à partir du point de coupure :

$$T_3 = 3 - 5 - 2 - 1 - 6 - 4 - 7 - 8$$

Autres opérateurs de mutation

Le même type de problèmes affecte l'opérateur de mutation. D'autres stratégies de mutation peuvent être mises en œuvre :

- Déplacement d'une seule ville.
- Échange de 2 villes.
- Échange de 2 segments.

Exemple d'un déplacement aléatoire : la ville 2 se déplace entre 8 et 5.

$$1 - 2 - 4 - 3 - 8 - 5 - 6 - 7 \rightarrow 1 - 4 - 3 - 8 - 2 - 5 - 6 - 7$$

Échange de 2 segments :

$$1 - 2 - 3 - |4 - 5| - 6 - |7 - 8| \rightarrow 1 - 2 - 3 - 7 - 8 - 6 - 4 - 5$$

Utilité des opérateurs génétiques

But : voir l'impact du croisement et de la sélection dans des cas simples.

Considérons un problème NK avec $N = 18$, $K = 3$, dont la solution optimale est par hasard la chaîne de bits $x = 000 \dots 000$. La mutation est toujours présente.

Le nombre moyen de générations nécessaires pour trouver la solution optimale (sur 100 exécutions) :

Fonction	avec croisement	sans croisement
Problème 1 (80 générations)	8	20
Problème 2 (80 générations)	8	48
Problème 3 (80 générations)	28	67
Problème 4 (200 générations)	65	127

Note : Parfois, la solution optimale n'est pas atteinte dans la limite du nombre maximum de générations (80 ou 200 ici). Dans de tels cas, l'exécution n'est pas incluse dans les statistiques.

Pourquoi le croisement est efficace ?

L'opérateur de croisement est **très efficace** avec les problèmes NK .

Si le point de croisement est k , nous avons :

$$x = x_1 x_2 \dots x_k | x_{k+1} \dots x_N$$

et la fitness est quasi-séparable :

$$\begin{aligned} f(x_1 x_2 \dots x_N) &= \sum_{i=1}^N h(x_{i-1}, x_i, x_{i+1}) \\ &= \sum_{i=1}^{k-1} h(x_{i-1}, x_i, x_{i+1}) + \\ &\quad h(x_{k-1}, x_k, x_{k+1}) + h(x_k, x_{k+1}, x_{k+2}) + \\ &\quad \sum_{i=k+2}^N h(x_{i-1}, x_i, x_{i+1}) \\ &= \text{Gauche}(x_1 \dots x_k) + \Delta(x_{k-1} \dots x_{k+2}) + \text{Droite}(x_{k+1} \dots x_N) \\ &\approx L + R \end{aligned}$$

Utilité de l'opérateur de croisement avec une fonction réelle

Cas trivial de la fonction à valeurs réelles, $f(x) = x$ pour $x \in [0, 1]$ encodée avec $N = 18$ bits.

$$x = \frac{\sum_{i=0}^{N-1} x_i 2^i}{2^N - 1}$$

Nombre moyen de générations requises pour atteindre l'optimum (limité à 80) :

Fonction	avec croisement	sans croisement
$f(x) = x$	47	54
$g(x) = -(x + 0.1)(x - 0.8) \sin(x/(6\pi))$	38	49

NB : Parfois avec $g(x)$ l'optimum n'est pas atteint dans les 80 générations.

Pour l'encodage des nombres réels, nous attendons un croisement neutre : les bits les plus significatifs sont les plus représentatifs de la qualité. Cependant, le croisement reste utile.

Exemple : Couverture optimale de zone par antennes GSM

Objectif : minimiser les zones de chevauchement. Solution basée sur une carte d'intensité, avec les positions des antennes sélectionnées.

Nous pouvons considérer des mutations (déplacer les antennes vers d'autres emplacements acceptables) et des croisements (fusionner des régions complémentaires dans deux cartes).

Dans cet exemple, la fitness est difficile à calculer, ce qui est souvent un problème clé commun dans les applications pratiques. La parallélisation peut aider.

Théorème du schéma (Schema Theorem)

But : Expliquer pourquoi les GA trouvent la solution.

Idée centrale :

- Les motifs de bits (c'est-à-dire les schémas) qui correspondent à une fitness élevée sont amplifiés au cours du processus évolutif.
 - La solution optimale est construite en concaténant ces “bons” schémas grâce à l'opération de croisement. On parle aussi de **blocs de construction** (building blocks).
 - Cependant, cela ne fonctionne pas toujours de cette façon, et le processus évolutif peut être “trompé” par des schémas sous-optimaux qui éloignent l'évolution.
-

Définitions

En pratique :

- Un **schéma** H est un modèle (ou patron) qui représente un sous-ensemble de chaînes de bits dans l'espace de recherche.
- Généralement écrit avec l'alphabet $\{0, 1, *\}$.
- Par exemple : $S = (1 * * 1 * 0)$ est un schéma avec 3 chiffres binaires fixes, et 3 chiffres binaires libres indiqués par $*$.
- Un individu correspond à un schéma (et réciproquement) s'il a les mêmes valeurs pour les positions fixes : (100100) correspond à $S = (1 * * 1 * 0)$.
- Un schéma représente un hyperplan dans l'espace de recherche.

Ordre et longueur d'un schéma

Deux quantités importantes caractérisent un schéma :

- **Ordre** $o(S)$ d'un schéma S : le nombre de positions fixes (c'est-à-dire non *). Pour $S = (1 * * 1 * 0) : o(S) = 3$.
- **Longueur définissante** $\delta(S)$ d'un schéma S : la distance entre la première position fixe et la dernière.

Exemples :

- $\delta(1 * * 1 * 0) = 6 - 1 = 5$
- $\delta(* * * 1 * *) = 4 - 4 = 0$
- $\delta(* 0 * 1 * *) = 4 - 2 = 2$

Pour un problème avec m bits :

- Un schéma d'ordre k correspond à 2^{m-k} chaînes.
- Une chaîne admet 2^m schémas : (0 symbole libre + 1 symbole libre + ... + m symboles libres) :

$$2^m = \sum_{k=0}^m \binom{m}{k}$$

Fitness d'un schéma S

Fitness d'un schéma : C'est la fitness moyenne de tous les individus de la population correspondant à S .

$$f_{P(t)}(S) = \frac{1}{M(S)} \sum_{i=1}^{M(S)} f(v_i)$$

où $v_1, \dots, v_{M(S)}$ sont les M individus de $P(t)$ satisfaisant le schéma S .

Fitness totale $F(t)$ de toute la population à l'itération t :

$$F(t) = \sum_{v \in P(t)} f(v)$$

Évolution du nombre moyen $M(t, S)$

Le théorème du schéma estime les variations du nombre moyen $M(t, S)$ d'individus correspondant au schéma S tout au long de l'évolution.

Nous avons :

$$M(S, t + 1) = M'(S, t) \times [\text{prob de survie après croisement}] \times [\text{prob de survie après mutation}]$$

où $M'(S, t)$ est le nombre de copies de S après sélection.

Si $M(S, t + 1) > M(S, t)$, alors le schéma S est amplifié. Sinon, il finira par disparaître.

Après sélection

En moyenne, le nombre $M'(S, t)$ d'individus correspondant au schéma S après **sélection** est :

$$M'(t, S) = n \frac{M(t, S)f(S, t)}{F(t)} = \frac{M(t, S)f(S, t)}{\bar{F}} \quad \text{avec} \quad \bar{F} = \frac{F}{n}$$

car nous avons n tirages, $M(t, S)f(S, t)$ est la probabilité totale du schéma, et $M(t, S)f(S, t)/F(t)$ est la probabilité de tirer un individu correspondant à S .

Un schéma dont la fitness est supérieure à la moyenne est amplifié.

Impact du croisement

Un schéma S est détruit par un croisement si le point de croisement tombe à l'intérieur du schéma. Avec m bits, il y a $m - 1$ points de croisement possibles, et la probabilité qu'il soit détruit est inférieure à $\frac{\delta(S)}{m-1}$ puisque, si les deux parents correspondent au schéma, il est préservé.

Compte tenu de la probabilité de croisement p_c , le schéma S survit avec une probabilité supérieure à $1 - p_c \frac{\delta(S)}{m-1}$.

Impact de la mutation

Si chaque bit de l'individu est inversé par mutation avec probabilité p_m , alors le schéma S survit avec une probabilité :

$$(1 - p_m)^{o(S)} \approx 1 - o(S)p_m \quad \text{si } p_m \ll 1$$

Inégalité fondamentale

En combinant les effets de la sélection, du croisement et de la mutation (avec $p_m \ll 1$) :

$$M(t+1, S) \geq \frac{M(t, S)f(S, t)}{F(t)} \left[1 - p_c \frac{\delta(S)}{m-1} - o(S)p_m \right]$$

Théorème du schéma de Holland

Théorème du schéma : Les schémas courts, de petit ordre ($\delta(S)$ et $o(S)$ petits) et avec une fitness supérieure à la moyenne sont amplifiés par le GA. Au fur et à mesure que le schéma est amplifié, sa fitness se rapproche de celle de la moyenne, et l'amplification ralentit.

Dans de nombreux cas, la solution optimale est obtenue en concaténant ces “bons” schémas courts, les soi-disant **blocs de construction**.

Mais dans d'autres cas, le GA peut être influencé par des schémas sous-optimaux, qui “trompent” le processus évolutif.

Population structurée

Jusqu'à présent, nous avons supposé que tous les individus d'une population donnée pouvaient interagir les uns avec les autres, sans limites. En biologie, ce sont des populations **panmictiques**.

Mais en biologie, les individus sont incapables de se déplacer arbitrairement loin chaque fois qu'ils se reproduisent. L'isolement de certaines sous-populations peut parfois avoir des effets surprenants (par exemple, Galapagos).

Nous pouvons donc utiliser cette inspiration biologique pour développer certaines idées sur les processus évolutifs métaheuristiques.

Il existe de nombreuses façons de structurer une population avec les EA. En pratique, nous choisissons :

- **Multi-populations**, ou populations insulaires.
- **Populations cellulaires**.

Le but n'est pas de simplement copier la nature. Nous observons que cette organisation est bénéfique à la performance de l'algorithme de recherche.

Multi-populations

Nous considérons un réseau de populations.

D'une part, ces populations évoluent selon l'EA. D'autre part, il y a des échanges d'individus entre les populations via les arêtes.

Par analogie avec la biologie, nous parlons de “**migrations**”.

Il est crucial de spécifier :

- Le nombre de sous-populations.
 - La topologie d'interaction entre les populations.
 - Le nombre d'individus par sous-population (traditionnellement maintenu constant).
 - Le nombre et le type d'individus migrants.
 - La fréquence de migration.
-

Choix classiques

- Garder les sous-populations relativement grandes (par exemple 50 individus).
 - Avoir de l'ordre de quelques dizaines de sous-populations.
 - Un anneau de connexion est souvent suffisant (dépendance faible de la topologie du réseau).
 - Entre 5% et 10% des individus émigrent. Les copies sont sélectionnées parmi les meilleurs individus.
 - Les individus immigrés remplacent les individus les moins aptes d'une sous-population donnée.
-

Avantages des multi-populations

- Convergence plus rapide pour le même effort de calcul.
- Nous maintenons une meilleure diversité grâce aux sous-populations séparées.
- Peut facilement être parallélisé (communications asynchrones).
- MAIS il y a un compromis entre le nombre de sous-populations et leur taille.

Comparaison entre différents groupements : 1 population avec 500 individus contre 50 populations avec 10 individus chacune. Le groupement optimal semble être autour de 10 populations avec 50 individus chacune.

Configuration	Taux de succès (sur 100 runs)	Fitness moyenne optimale
1-500	53	4.964
2-250	55	4.899
5-100	64	4.778
10-50	76	4.208
50-10	62	4.770

E représente l'effort de calcul.

Populations cellulaires

Cela consiste à organiser les individus sur une grille régulière (ou dans certains cas particuliers sur un graphe).

Les opérations de sélection et de croisement impliquent un individu et ses voisins spatiaux (ou voisins topologiques dans le cas d'un graphe).

La **sélection par tournoi** est la plus courante : l'individu peut être remplacé par le voisin le plus apte. Un tournoi binaire entre l'individu et un voisin choisi au hasard est également possible.

Le croisement est effectué avec probabilité p_c avec un voisin choisi au hasard.

Mutation : inchangée.

Mise à jour parallèle.

Avantages des populations cellulaires

L'avantage principal de l'approche cellulaire est que la sélection est moins rapide que pour une population panmictique : meilleure exploration spatiale.

Cela peut être vérifié en considérant le temps de prise de contrôle. L'individu le plus apte "se propage" mieux dans son voisinage.

Pour une organisation spatiale basée sur un anneau, la vitesse de propagation est linéaire, et basée sur une grille, elle est quadratique.

La vitesse de sélection peut donc être ajustée en modifiant la taille/topologie du voisinage.

Temps de prise de contrôle avec populations structurées

Topologie	Temps de prise de contrôle τ
Anneau	$\mathcal{O}(n)$
Grille	$\mathcal{O}(\sqrt{n})$
Uniforme (panmictique)	$\mathcal{O}(\log n)$